

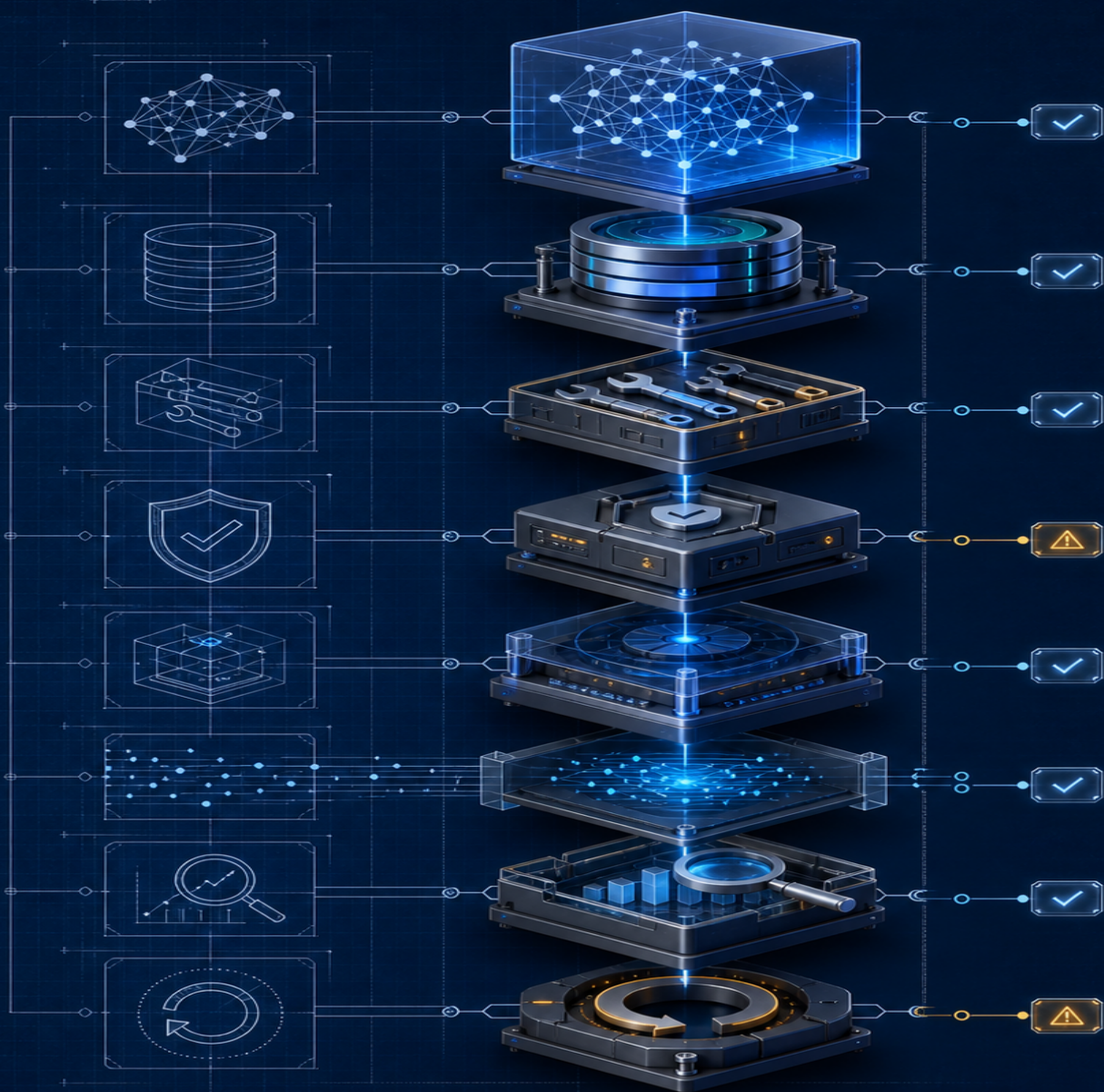
WGP AGENT BODIFICATION FLOW

WGP Agent Bodification Flow (WGP-ABF)

Agent Blueprints, Evidence-Backed Evaluation, and Validated Component Replacement

Version 0.4 • Bilingual Illustrated Public Draft

Originator and Principal Author: Wang Guangping



Contents

This edition is a public draft. Repository schemas and conformance tests define machine-readable semantics.

Abstract	4
1. Vision, Intended Users, and Non-goals	5
1.1 Vision	5
1.2 Intended Users	6
1.3 Non-goals	6
2. Problem Boundary and Design Principles	6
2.1 Seven Problems to Solve	6
2.2 Six Design Principles	7
3. Normative Language, Versioning, and Conformance Levels	8
3.1 Versioning	8
3.2 Tiered Core Conformance and Optional Projection Conformance	8
4. Normative Model: ABIR, Recipe, Evidence, and Projections	9
4.1 Core Object Categories	10
4.2 Canonicalization and Identity	10
5. Structural Provenance: Fidelity Is Not Authority	10
5.1 Operation Capabilities	11
6. Strongly Typed Graphs, Ports, Modules, and Policies	12
7. RecipeDiff, Approval, Transactions, and Drift	13
8. Runtime Events, Causal Ordering, and Three Replay Modes	14
9. Observability and Open Mappings	15
10. Evaluation Design, Statistical Protocol, and Attribution Limits	16
10.1 Units of Evaluation	16
10.2 Evidence Designs	16

10.3 Minimum Statistical Record 16

11. Diagnostic Hypotheses, Intervention Validation, and Improvement Decisions 17

12. A Progressive Experience for General Users 18

12.1 Goal Layer 18

12.2 Blueprint Layer 18

12.3 Expert Layer 18

13. Two-Dimensional Blueprints, Body Projections, and Accessibility 18

14. Threat Model, Supply Chain, and Data Governance 19

15. Versioned DeepSeek Harness Adapter Case 20

16. Implementation Roadmap and Acceptance Criteria 20

17. Open Ecosystem, Governance, and Intellectual Property 21

Appendix A: Minimum Interoperable Objects 22

Appendix B: Claim Maturity 23

Appendix C: Normative Release Checklist 23

References 24

Revision Notes 24

Abstract

Modern agents combine models, tools, memory, sandboxes, permissions, approvals, subagents, runtimes, and external services. Generative programming is lowering implementation cost, but the engineering assets that require long-term maintenance now extend beyond source code to assembly recipes, evaluation sets, evidence, and governance policies. Agent configuration, however, still lacks the semantic diffs, review, testing, attribution, and rollback mechanisms long available to source code.

WGP-ABF proposes a platform-neutral method for agent configuration engineering. It turns configuration into an artifact that can be deconstructed, edited, executed, evaluated, and diagnosed. The method defines three protocols and one blueprint discipline: the structure protocol uses WGP-ABIR and [AssemblyRecipe](#) to express canonical structure; the runtime protocol represents execution with events whose storage, origin, and replay roles are explicit; the evidence protocol constrains improvement claims through layered evaluation, statistical conditions, and diagnostic reports; and the two-dimensional engineering blueprint is the authoritative human editing projection of ABIR. Body and three-dimensional views are optional, read-only explanatory projections.

The primary value of WGP-ABF is not reducing configuration input. It is improving understanding and attribution: reveal what a configuration actually assembles, locate weak components or relationships, express a change as a reviewable [RecipeDiff](#), and test whether that change improves results on the same task set under controlled conditions. A Core-conformant implementation MUST identify structural provenance, declare operation capabilities independently, separate the three event axes, make every score traceable to source evidence, and prove that a blueprint edit can change the real runtime. DeepSeek Harness (DSH) is the first version-pinned adapter case, but it does not define the conceptual limits of the method.

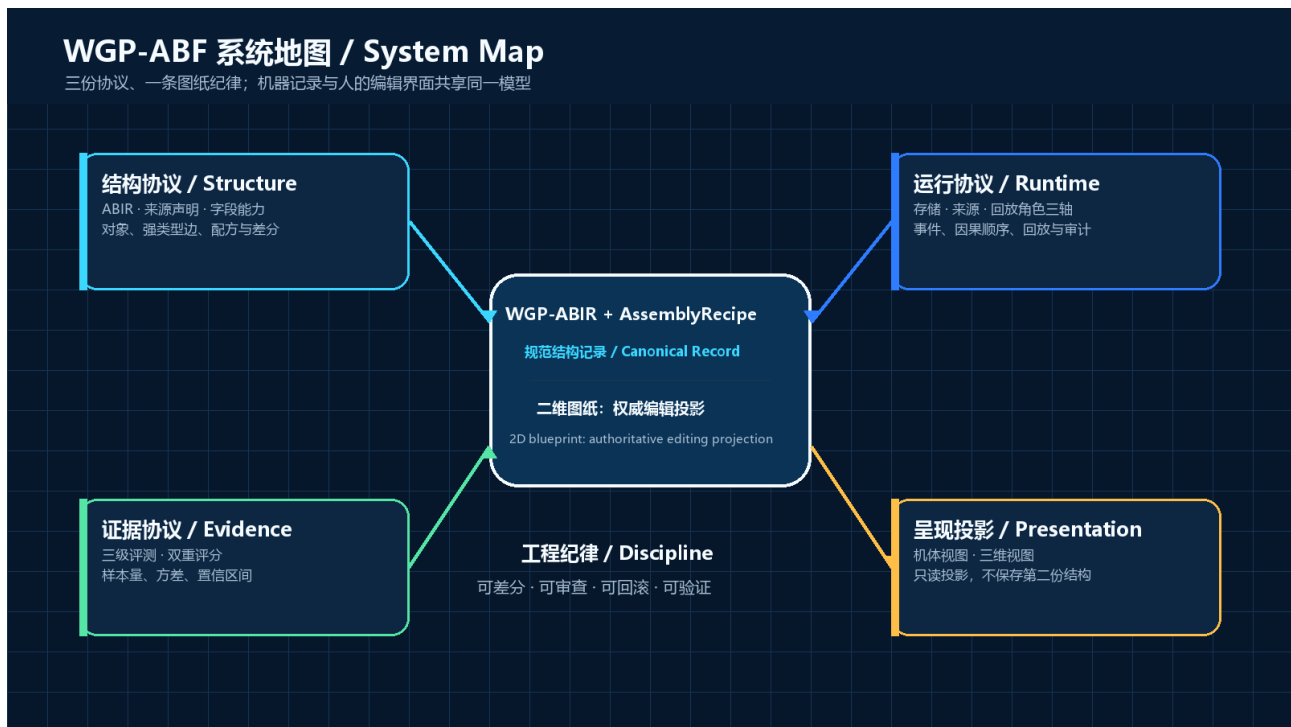


Figure 1 — System map. WGP-ABIR and `AssemblyRecipe` are the canonical, versioned structural record; the two-dimensional blueprint is their authoritative human editing projection. Every view **MUST** share this model and **MUST NOT** maintain a hidden second copy of the business structure.

1. Vision, Intended Users, and Non-goals

1.1 Vision

WGP-ABF aims to let general users understand an agent as naturally as they would read an assembly drawing, let developers review configuration as rigorously as they review code, and let teams validate every modification as a controlled experiment.

It establishes one continuous workflow:

1. import configuration from a real platform and identify the source of every item;
2. present components, interfaces, relationships, permissions, and containers in a two-dimensional engineering blueprint;
3. express each modification as a readable, reviewable, and applicable diff;
4. compile to the target platform and detect drift;
5. collect runtime evidence under repeatable conditions; and
6. reach a bounded conclusion: promote the change, retain it as an experiment, or roll it back.

1.2 Intended Users

- General users who want to understand, modify, or combine agents.
- Product teams building Agent Studios, operations platforms, and visual editors.
- Security and governance teams that audit permissions, extensions, and data flows.
- Engineers studying agent evaluation, diagnosis, and component interaction.
- Open-source communities that want to publish portable configurations or compatible adapters.

1.3 Non-goals

WGP-ABF does not promise to unify the execution semantics of every platform. It does not treat a humanoid appearance as structural truth, use architecture hiding as a substitute for security controls, claim that a single A/B comparison establishes causal attribution, or designate any one model or framework as the only valid implementation.

2. Problem Boundary and Design Principles

2.1 Seven Problems to Solve

- 1. Invisible structure:** users see YAML, JSON, or forms, but not the complete assembly.
- 2. Poor comparability:** equivalent concepts across platforms lack shared structural semantics.
- 3. Poor reviewability:** a text diff does not reliably express which component changed or which path is affected.
- 4. No reliable write-back:** many visualizations are read-only, and their edits cannot safely return to a real runtime configuration.
- 5. Ambiguous replay:** incomplete events and unclear retention semantics blur the difference between View Replay and real Re-execution.
- 6. Weak attribution:** scores exist without a reference chain connecting them to source evidence and structural objects.
- 7. Weak governance:** extension provenance, permissions, approval, compensation, and version drift lack a unified record.

2.2 Six Design Principles

- **Reality first:** every structural claim **MUST** state its provenance and coverage.
- **One model, multiple projections:** the machine-readable canonical record, 2D editing blueprint, and 3D presentation do not duplicate business facts.
- **Authority is separate from provenance:** F3–F0 does not automatically grant view, edit, compilation, or write-back authority.
- **Declaration is separate from state:** the recipe describes desired configuration; runtime events describe what actually happened.
- **Evidence precedes promotion:** a modification enters a stable version only after satisfying predeclared gates.
- **Progressive complexity:** first-time users complete safe goals before experts expand fields, statistics, and governance details.

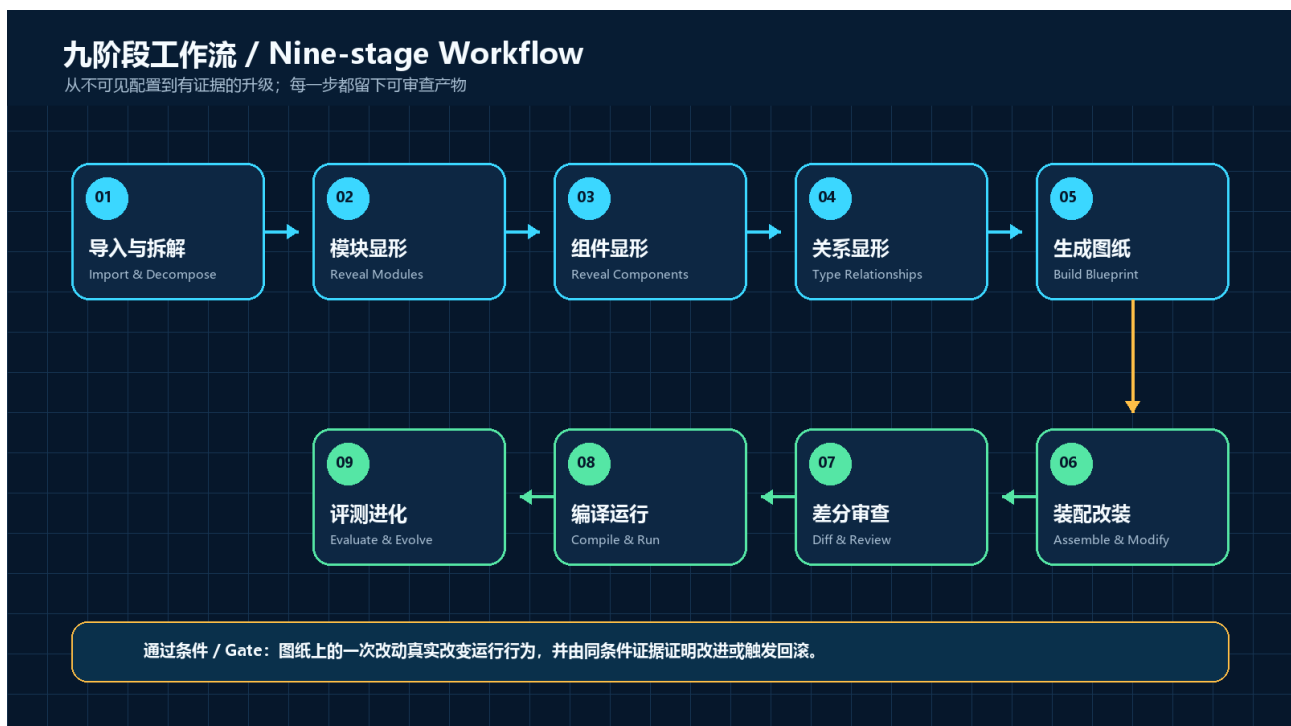


Figure 2 — Nine-stage workflow. Every stage leaves a reviewable artifact. The final gate is not merely “the agent started” ; the modification must change the real runtime and be supported by comparable evidence, or trigger rollback when the gate is not met.

3. Normative Language, Versioning, and Conformance Levels

The Chinese terms corresponding to requirement, prohibition, recommendation, and permission are aligned with the RFC-style key words **MUST**, **MUST NOT**, **SHOULD**, and **MAY**. In this English edition, only sentences that explicitly use these uppercase words are normative; all other text is explanatory or advisory.

3.1 Versioning

- `specVersion`: the semantic version of the WGP-ABF machine specification.
- `schemaVersion`: the semantic version of one JSON Schema.
- `recipeVersion`: the monotonic version of one assembly recipe.
- `definitionVersion`: the version of a component definition.
- `adapterVersion`: the version of a platform adapter.

The whitepaper version **0.4** is a reader-facing version-series label. npm packages, validators, and machine-specification releases use SemVer **0.4.0**. Format values such as `wgp-abir/0.4`, `wgp-recipe-diff/0.4`, and `wgp-runtime-event/0.4` identify the **0.4 format compatibility family**, not an npm package version. Patch releases within a compatibility family **MUST** continue to accept existing valid documents; a breaking change requires a new format family. The immutable Git tag `v0.4.0` pins the Schemas, examples, and build artifacts for this release. Schema `$id` values **MUST** reference that tag rather than a movable branch.

Versions in the `0.y.z` range are public drafts. A breaking field or semantic change **MUST** increment the minor version; a patch release **MUST NOT** change the meaning of an existing valid document. Whitepaper 0.4, package and specification version 0.4.0, the `/0.4` format family, and Git tag `v0.4.0` refer to the same initial semantic baseline, but serve different purposes: document identification, tool distribution, compatibility decisions, and immutable release location. They are not interchangeable.

3.2 Tiered Core Conformance and Optional Projection Conformance

Level	Name	Minimum requirements
C0	Visual Explainer	A teaching or presentation prototype clearly labeled non-conformant; it MUST NOT claim runtime fidelity

Level	Name	Minimum requirements
C1	Blueprint Conformance	Valid ABIR + AssemblyRecipe, strongly typed edges, per-object provenance, one canonical record, and an exportable representation
C2	Operational Conformance	C1 + field-level capabilities, semantic diff, risk review, native compilation, runtime-binding verification, and rollback discipline
C3	Evidence Conformance	C2 + three event axes, safe replay, evaluation uncertainty, claim maturity, object-level diagnosis, and comparable re-evaluation

An implementation **MUST** publish its validator version, supported Schemas and adapter versions, exceptions, and evidence supporting its conformance claim. A polished body or 3D view does not increase Core Conformance.

Projection Conformance is a separate, optional label. If an implementation provides 2D, body, or 3D projections, it **MUST** prove that each projection is generated from the canonical record, restrict editing to 2D fields with declared write-back support, and provide keyboard operation, text alternatives, and non-color status encoding. The absence of a body or 3D projection does not affect C1–C3.

4. Normative Model: ABIR, Recipe, Evidence, and Projections

The sole canonical structural record in WGP-ABF is formed jointly by the **WGP Agent Blueprint Intermediate Representation (WGP-ABIR)** and [AssemblyRecipe](#):

- **WGP-ABIR** represents objects, ports, relationships, source claims, and available operation capabilities.
- **AssemblyRecipe** represents compilable desired configuration, version anchors, and the target platform.
- **Evidence Store** preserves events, task results, artifact references, and evaluation results.
- **ProjectionProfile** preserves layout, themes, body mappings, and accessibility descriptions.

[ProjectionProfile](#) **MUST** remain separate from structural objects and **MUST NOT** become a required field of a [Component](#). Visual identifiers such as [arm.browser](#) **MUST NOT** enter the core type system. A layout-only change **MUST NOT** change recipe semantics.

4.1 Core Object Categories

Category	Purpose	Examples
Component	A replaceable, configurable, independently observable runtime unit	Model provider, memory retriever, tool executor
Resource	Data or capability resources read or written by components	Vector store, file system, model credential reference
Policy	Constraints enforced outside the model	Network policy, approval rule, budget limit
Artifact	Addressable runtime or evaluation output	Patch, report, screenshot, log excerpt
Interface	Port, protocol, or service definition	Chat completion, tool call, RPC
Container	A unique containment hierarchy over objects	Subagent module, tool group

Messages, prompts, task definitions, and policies SHOULD NOT all be disguised as [Component](#) merely because doing so is convenient for drawing.

4.2 Canonicalization and Identity

A conformant implementation MUST assign stable, namespaced IDs to objects. JSON used for hashing, signatures, or caching SHOULD use deterministic canonicalization; RFC 8785 JSON Canonicalization Scheme is recommended. Containment MUST have one authoritative representation and MUST NOT be maintained simultaneously through [Container.contains](#) and duplicate containment edges.

Cross-recipe references MUST pin a version or content digest. Import MUST fail with a localized diagnostic when a reference cannot be resolved, an undeclared recursive cycle is present, or IDs collide.

5. Structural Provenance: Fidelity Is Not Authority

F3–F0 remains a clear interface badge, but [structuralProvenance](#) describes only the structural source of one claim. It is not a trust conclusion and grants no operation authority:

- **F3 Native:** obtained from a native, locatable, authoritative structural representation of the target platform.

- **F2 Exported:** obtained from a repeatable structural export produced by the platform or a versioned adapter.
- **F1 Inferred:** inferred through static analysis, runtime observation, or other evidence.
- **F0 Manual:** entered directly by a person without a higher-grade structural source.

In addition to the general claim identifier `id`, every `SourceClaim` MUST contain `subjectId`, `fieldScope`, `structuralProvenance`, `identityAssurance`, `fieldCoverage`, `runtimeEvidence`, `operationCapabilities`, `trustState`, `method`, and `observedAt`. `fieldScope` is the set of JSON Pointers covered by the claim. Identity stability, field coverage, runtime evidence, and trust state MUST be recorded separately; none may be inferred from the F level. `adapter`, `evidenceRefs`, and `expiresAt` are optional. An expired claim MUST NOT continue to serve as current evidence.



Figure 3 — Provenance is separate from authority. F3 Native does not imply editability, and F2 Exported does not imply “limited editability.” Operation capabilities require independent adapter and field-level evidence.

5.1 Operation Capabilities

`operationCapabilities` is the composable capability set that a `SourceClaim` assigns to all fields covered by its `fieldScope`. It MAY contain any compatible combination of:

- `view`: the fields can be displayed reliably;
- `edit`: a valid modification can be generated;
- `compile`: the fields can be compiled into target-platform configuration;
- `roundTrip`: export, modification, and write-back preserve the declared semantics;

- `hotReload`: the fields can take effect during a declared runtime phase without restarting the whole assembly; and
- `replace`: the target can be atomically replaced while preserving interface constraints.

These values describe technical capabilities proved by an adapter. They do not represent the current user's authorization, deployment policy, or approval outcome; an operation still requires all three. A capability **MUST NOT** be inferred from `structuralProvenance`, `trustState`, or a single runtime observation. The assembly interface **SHOULD** report a claim-coverage vector such as `F3 62% / F2 21% / F1 12% / F0 5%` rather than hide the distribution behind one minimum value. A critical path **MUST** be declared for a specific task.

6. Strongly Typed Graphs, Ports, Modules, and Policies

Structural edges **MUST** connect explicit ports rather than merely connect two boxes:

```
sourceComponent + sourcePort
-> edge(type, cardinality, schema, ordering, policyRefs)
-> targetComponent + targetPort
```

At minimum, a port declares direction, data Schema, single- or multi-value cardinality, and synchronous or streaming semantics. A compiler **MUST** reject direction conflicts, incompatible data Schemas, unconnected required ports, and unauthorized permission expansion.

Recommended semantic relationship labels include:

- `data.flow`: a message or structured-data flow;
- `control.invoke`: an invocation or control relationship;
- `capability.consume`: a component consuming a capability;
- `policy.enforce`: a policy enforced on an object or edge;
- `artifact.produce`: a component producing an addressable artifact; and
- `evidence.support`: evidence supporting a claim or score.

These profile-level labels **MUST** map explicitly to the allowed core `edgeKind` values in the ABIR Schema; they do not extend that enum implicitly. `errorPropagation` belongs to a runtime or diagnostic projection and **MUST NOT** be conflated with a static structural edge.

A reusable subgraph is represented by a `ModuleDefinition`. A module instance **MUST** declare exposed ports, parameters, its internal version, and state-isolation semantics. A recursive module **MUST** declare depth, resource, and termination limits.

7. RecipeDiff, Approval, Transactions, and Drift

A modification **MUST** first become a `RecipeDiff`; it **MUST NOT** overwrite a file directly. In addition to format, identity, creation time, and recipe identity, its core fields are:

- `baseVersion` and `baseContentHash`: pin the recipe version and canonical content digest before the change;
- `targetVersion` and `targetContentHash`: pin the intended version and content digest after the change;
- `preconditions`: declare transaction-level recipe-version, content-hash, path-existence, or path-value preconditions;
- `operations`: an ordered list of atomic operations, each with its own `precondition` and `restartRequired`, plus an executable configuration inverse in `operations[].compensation`;
- `riskAssessments`: per-path risk category, severity, mitigation, residual risk, and risk approval;
- `compensation`: the transaction recovery strategy and, under `externalSideEffects`, the action, verification, idempotency, and owner for compensating each external effect; and
- `approval`: whether the overall transaction requires approval and its state; this does not replace the risk-specific approval in `riskAssessments[].approval`.

An operation-level configuration inverse, recipe rollback, compensation for an external side effect, and a genuinely irreversible action are four distinct concepts. `operations[].compensation` describes only the inverse operation on one `AssemblyRecipe` path. Recipe rollback applies those inverses in a safe order and verifies the target digest. Top-level `compensation.externalSideEffects` handles effects that have already occurred outside the recipe document. An action with neither a real inverse nor reliable compensation is irreversible and **MUST NOT** be disguised as recoverable with a no-op. Restoring an older configuration cannot unsend an email, retract already published content, or restore data deleted from an external system.

The compiler **MUST** be deterministic and idempotent, and **MUST** verify the prerequisite version and digest before applying a diff. Atomic application either commits every operation or leaves the target unchanged. A target platform without atomic application **MUST** report its partial-failure semantics and recovery points explicitly. The system **SHOULD** re-import before and after application to detect semantic drift.

Irreversible operations **MUST** be denied by default. The Core Schema cannot label a no-compensation action “safely recoverable.” A deployment extension **MAY** allow it only under explicit policy, separate `riskAssessments` disclosure, independent approval, and an audit record naming the non-reversible result.

8. Runtime Events, Causal Ordering, and Three Replay Modes

At minimum, the `RuntimeEvent` envelope contains:

```
{
  "format": "wgp-runtime-event/0.4",
  "eventId": "44444444-4444-4444-8444-444444444444",
  "runId": "run.example-01",
  "sessionId": "session.example-01",
  "sequence": 12,
  "eventType": "tool.completed",
  "occurredAt": "2026-08-26T08:00:00Z",
  "observedAt": "2026-08-26T08:00:00.042Z",
  "producer": { "name": "adapter.dsh", "version": "0.4.0", "instanceId": "runtime.local-01" },
  "subject": { "abirId": "agent.example", "objectId": "component.tool.browser", "recipeId": "recipe.example" },
  "trace": { "traceId": "0123456789abcdef0123456789abcdef", "spanId": "0123456789abcdef" },
  "storageClass": "durable",
  "origin": "primary",
  "replayRole": "expectedOutcome",
  "payload": {},
  "redaction": { "state": "none", "removedFields": [] }
}
```

The three event axes **MUST** remain independent:

Axis	Values	Meaning
<code>storageClass</code>	<code>durable</code> / <code>ephemeral</code>	Whether the event enters a durable record or exists only in the live session
<code>origin</code>	<code>primary</code> / <code>derived</code>	Whether this is a primary observation or a deterministic derivative of other events
<code>replayRole</code>	<code>input</code> / <code>expectedOutcome</code> / <code>verificationOnly</code> / <code>projectionOnly</code> / <code>excluded</code>	Whether the declared replay consumes, compares, verifies, projects, or excludes the event

A derived event may be durable, and an ephemeral event may be a primary observation. If `tool.started` records an actual start time, it is a primary event; it does not become derived merely because the interface could infer it.

An implementation **MUST** distinguish:

- 1. View Replay:** reconstructs the interface using recorded facts only;
- 2. Simulation Replay:** validates interfaces and paths with fixed dependencies or a recorded model stream; and

- 3. Re-execution:** invokes real components again, so results may change and a new event chain is produced.

Factual replay input comes from `durable + primary` events or access-controlled encrypted references to them. A derived layer **MUST** be reproducible by its declared deterministic function and, where represented as a derived event, use the Schema's `derivation.sourceEventIds`. Event processing **MUST** also define policies for duplicate delivery, out-of-order and late events, clock skew, persistence failure, and checkpoints before external side effects.

Privacy takes priority over preserving every plaintext payload. A system **MAY** retain an input manifest, digest, or access-controlled encrypted reference, but it **MUST** state retention, deletion, key-management, and tenant-isolation semantics.

9. Observability and Open Mappings

WGP-ABF does not replace OpenTelemetry. Implementations **SHOULD** align `traceId` / `spanId` with OpenTelemetry traces and carry WGP object IDs, recipe digests, and evidence references as semantic attributes. Generative AI semantic conventions continue to evolve; an adapter **MUST** pin the version or commit it implements and **MUST NOT** claim to use an unspecified "latest" version.

Telemetry collection **MUST** follow these rules:

- keep normalized-event namespaces separate from native platform-event namespaces;
- publish and version transformation rules, including any semantic loss;
- redact sensitive fields by default, and never place secret values in ordinary logs;
- preserve explicit references when events and structural objects have many-to-one or one-to-many relationships rather than inventing one-to-one identity; and
- let scores and diagnoses reference stable evidence IDs instead of copying drift-prone log text.

10. Evaluation Design, Statistical Protocol, and Attribution Limits

WGP-ABF records two orthogonal dimensions: the **unit of evaluation** and the **evidence design**.

10.1 Units of Evaluation

- `eval.component`: validates one component's interface, local quality, cost, and safety.
- `eval.assembly`: validates interactions within a module or subgraph.
- `eval.agent`: validates complete-agent task outcomes and user experience.

A component MAY have both a **Standalone Performance Score** and an **In-Assembly Contribution Score**. They are not interchangeable.

10.2 Evidence Designs

1. **Contract regression**: fixes a model stream or dependency to validate interfaces, events, and runtime paths.
2. **Paired behavioral evaluation**: uses paired real samples on the same tasks and environment to estimate a behavioral difference.
3. **Factorial or interaction evaluation**: uses a factorial design when multiple components interact, or explicitly declines single-component attribution.
4. **Observational diagnosis**: finds associations and proposes hypotheses; it MUST NOT present them as validated root causes.

A fixed model stream can reduce model-output variation, but cannot make whole-system variance “near zero” and cannot by itself prove that a component caused a change in final quality. If a modification changes context, tool results, or call IDs, the old stream may no longer be valid.

10.3 Minimum Statistical Record

Every comparable Scorecard MUST record metric definitions, the independent sampling unit, task-set version, pairing method, repetition count, effect size, variance estimate, confidence-interval method, significance level, prespecified power, minimum detectable effect (MDE), outlier and missing-data rules, stopping rule, evaluator version, and inter-rater agreement for human labels.

Safety, privacy, and authorization measures are gates that an aggregate quality score cannot offset. Cross-platform behavioral scores MAY be compared, but component-level attribution MUST show the claim coverage and evidence strength of both platforms.

11. Diagnostic Hypotheses, Intervention Validation, and Improvement Decisions

A diagnostic report MUST NOT label a correlated observation directly as `rootCause`. It uses:

- `suspectedCause`: supported by evidence but not yet tested by an intervention;
- `testedCause`: evaluated through a preregistered intervention;
- `validatedCause`: the effect passes the gate and alternative explanations are controlled;
- or
- `rejectedCause`: the intervention does not support the hypothesis.

Every diagnosis MUST reference a `componentId`, `edgeId`, or subgraph, symptom events, comparison evidence, and uncertainty. A change decision has exactly four states: `promote`, `hold`, `reject`, or `rollBack`. “No significant decline detected” MUST NOT be rewritten as “improved.”

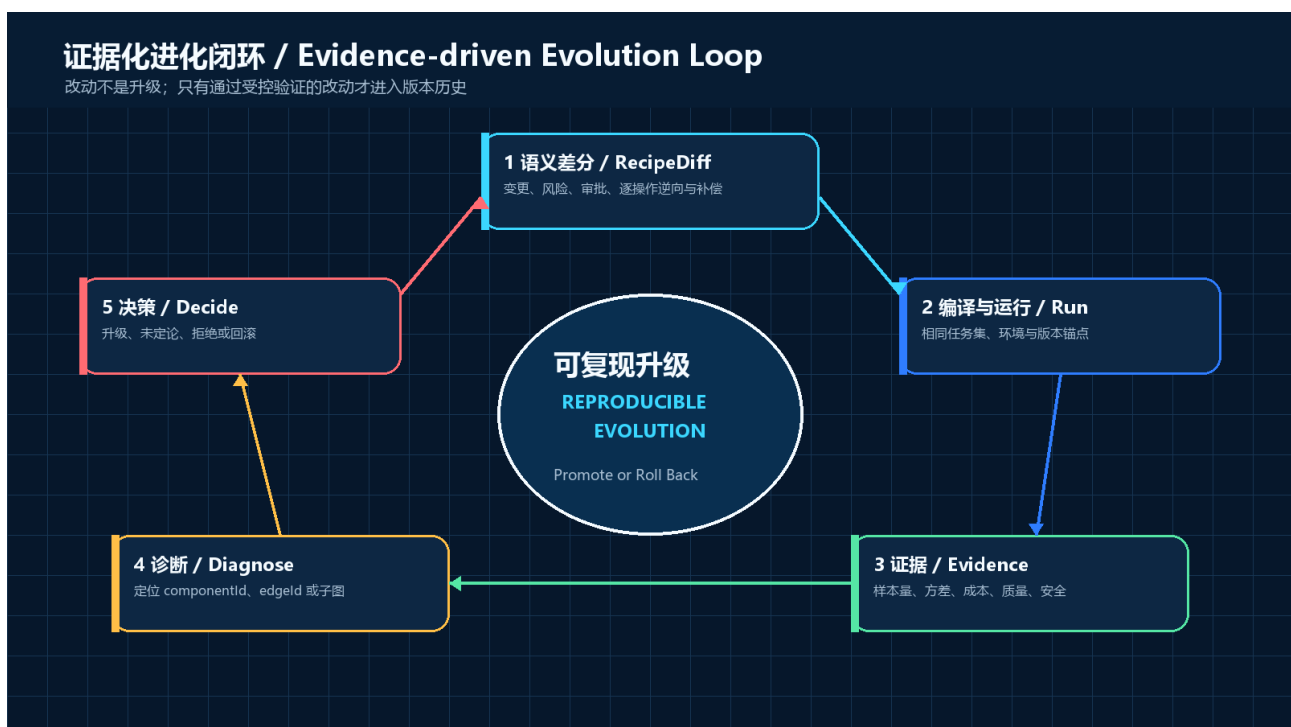


Figure 4 — Evidence-backed improvement loop. “Evolution” in this document means a reproducible improvement process controlled by a person or governance policy. It does not mean the system may rewrite itself without authorization.

12. A Progressive Experience for General Users

Visualization does not mean placing every field on a canvas. A conformant product experience SHOULD provide three layers:

12.1 Goal Layer

The user selects a goal such as “use the network more safely,” “add long-term memory,” “change the model,” or “reduce cost.” The system explains the expected change, its risks, and whether a restart is required in plain language.

12.2 Blueprint Layer

The user sees components, relationships, and permissions. A guided flow handles safe preset modifications. Before requesting confirmation, every change shows its semantic diff, affected scope, and recovery capability.

12.3 Expert Layer

Developers can expand port Schemas, [SourceClaim](#), raw events, statistical plans, compiler diagnostics, and native platform configuration. Expert details and first-time-user views reference the same object IDs.

A beginner mode MUST NOT use animation to hide real waiting, or present a high-risk action as an ordinary switch. An error message MUST identify the failed object, available actions, and whether the current configuration changed.

13. Two-Dimensional Blueprints, Body Projections, and Accessibility

The two-dimensional engineering blueprint is the authoritative editing surface, while ABIR and [AssemblyRecipe](#) remain the canonical record. Every canvas edit MUST generate a [RecipeDiff](#). A field without verified write-back MUST remain read-only and explain why.

Humanoid, photorealistic, anime, pixel-art, or 3D appearances can help general users understand an agent, but they belong only in [ProjectionProfile](#). A component may map to

different visual regions in different themes; a projection **MUST NOT** change the component type in reverse.

At minimum, projections **SHOULD** support:

- keyboard-only selection, inspection, diff review, and application;
- risk and status encoding that does not depend on color;
- text alternatives for nodes, relationships, and charts;
- zoom, reduced motion, and high contrast; and
- a 2D or list-based path for every function when 3D is unavailable.

14. Threat Model, Supply Chain, and Data Governance

Security boundaries in WGP-ABF **MUST** be enforced outside the model by policy engines, the operating system, sandboxes, and server-side permissions. Hiding ABIR is not a substitute for a security control.

A Core-conformant deployment **SHOULD** address:

- prompt injection and indirect prompt injection;
- malicious extensions, dependency compromise, and supply-chain substitution;
- server-side request forgery, cross-site scripting, and uncontrolled network access;
- secret or log leakage and cross-tenant data access;
- tampering with or replay of blueprints, recipes, events, and telemetry; and
- approval bypass, privilege escalation, and irreversible external side effects.

Components and recipes **SHOULD** record provenance, license, digest, signature or attestation, SBOM reference, and revocation state. By default, an agent **MUST** be limited to a minimized, redacted capability manifest. Reading the complete authority graph requires an independent permission check, audit record, and tenant-isolation policy.

Any modification that changes network, file, credential, external-publishing, or billing capability **MUST** be displayed as a separate high-risk capability in the diff.

15. Versioned DeepSeek Harness Adapter Case

This section presents only a reviewable reference mapping. It does not imply that every DSH object is inherently F3, or that WGP-ABF is subordinate to DSH.

Verification anchors for this edition:

- repository: [pingta-guangpingwang/deepseek-harness](#);
- locally verified commit: [0672d5ddfaf7d675d8e8bb37f69072bd98b5b0f7](#);
- nearest tag relationship: the verified commit is one commit after [dsh-v0.1.1-rc.2](#), described as [dsh-v0.1.1-rc.2-1-g0672d5ddf](#); and
- verification date: August 26, 2026.

DSH provides a strong foundation for declared-configuration and event adapters through `--dump-config`, generated configuration, tool, and module catalogs, and the durable `SessionEvent` catalog. Configuration rows, service registrations, tool contributions, and runtime entities may still have one-to-many relationships, so an adapter MUST establish a binding for each object or claim. Only fields that can be reconstructed from declared configuration and pass adapter tests MAY be marked F3 with `roundTrip` capability.

DSH `SessionEvent` and live Cordis events are separate event spaces; mappings MUST preserve their namespaces. `assistant/chunk` is a streaming fragment, not a complete `model.completed` event, and `turn/end` does not necessarily mark the end of an entire run. Hot reload MUST be described by a component's `reloadStrategy`, in-flight behavior, and restart conditions.

A reference compiler can compile the write-back-capable ABIR subset into a profile patch. Evidence, layout, inferred relationships, and diagnostic results MUST NOT be written back incorrectly as native platform configuration.

16. Implementation Roadmap and Acceptance Criteria

The nine-stage workflow in Figure 2 is implemented through the single milestone table below. Other documents SHOULD reference this table and MUST NOT maintain a second P0–P5 definition.

Stage	Deliverable	Acceptance criterion
P0 Specification Foundation	Core Schemas, terminology, valid and invalid examples	Schemas are executable; Chinese and English object names align
P1 Import and Read-only 2D	At least one version-pinned adapter, source claims, and 2D rendering	Unsupported semantics and information loss remain visible
P2 Events and Replay	Event envelope, View Replay, and evidence references	Policies for disorder, duplicates, missing events, and privacy pass tests
P3 Diff and Write-back	RecipeDiff, dry run, approval, compilation, and drift detection	One blueprint edit changes real configuration and can be recovered safely
P4 Evaluation and Diagnosis	Three evaluation units, Scorecard, hypothesis states, and decision	Statistical plan is preregistered; every score resolves to evidence
P5 Optional Projections and Ecosystem	Body/3D views, compatibility suite, and community adapters	Core functions do not depend on 3D; independent implementations interoperate

A stage **MUST NOT** satisfy its acceptance criteria by fabricating F3 provenance, dropping unfavorable runs, changing the control task set, or using an aggregate score to conceal a security regression.

17. Open Ecosystem, Governance, and Intellectual Property

WGP-ABF was conceived and first authored by Wang Guangping in 2026. Version 0.4 uses a file-scoped dual-license model:

- whitepapers, README files, governance documents, and original diagrams: **Creative Commons Attribution 4.0 International (CC BY 4.0)**;
- Schemas, examples, build scripts, and reference code: **Apache License 2.0**; and
- the WGP-ABF name, logo, and any future compatibility or certification marks: not granted by those licenses and governed by [TRADEMARKS.md](#).

The open licenses do not transfer the author’ s copyright in the original work. They grant the public permission to copy, modify, and use the work subject to their terms. Copyright protects the text, diagrams, and specific expression in this document; it generally does not create an exclusive right over an abstract idea, method, or process. Brand protection, patent eligibility, and jurisdiction-specific rules require separate professional legal advice.

Normative semantic changes SHOULD proceed through a public RFC. Every breaking change MUST update the Schemas, valid examples, invalid examples, migration notes, and both language editions together. Before v1.0, if a Chinese-English difference cannot be reconciled immediately, the original Chinese edition is the temporary interpretive source and a public translation issue MUST be opened. Machine-verifiable semantics are governed by the Schemas and conformance tests.

Canonical repository: [pingta-guangpingwang/wgp-agent-bodification-flow](https://github.com/pingta-guangpingwang/wgp-agent-bodification-flow). Specification defects, interoperability problems, translation differences, and change proposals SHOULD be filed in the public [Issues](#), citing the affected format compatibility family, Schema `$id`, or immutable Git tag.

Appendix A: Minimum Interoperable Objects

A Core Conformance document contains at least:

1. **AbirDocument**: specification version, objects, ports, edges, [SourceClaim](#) records, and projection references;
2. **AssemblyRecipe**: target platform, desired configuration, version anchors, and compilation capabilities;
3. **RecipeDiff**: [baseVersion](#), [baseContentHash](#), [targetVersion](#), [targetContentHash](#), [preconditions](#), [operations](#), [riskAssessments](#), [compensation](#), and [approval](#);
4. **RuntimeEvent**: causal identity, the three event axes, object references, and artifact references; and
5. **EvaluationRun**: task set, environment, comparison, metrics, statistical plan, and evidence.

The repository provides JSON Schemas in [spec/](#) and a minimum closed-loop example in [examples/minimal-agent/](#). If a Schema and this document conflict, the conflict MUST become a specification Issue; an implementation MUST NOT choose one silently.

Appendix B: Claim Maturity

Level	Permitted wording	Prohibited wording
Observation	"Co-occurs with the failure" ; "warrants investigation"	"Caused the failure"
Regression	"Preserves/breaks this interface path"	"Improves final agent quality"
Controlled comparison	"The average difference under these tasks and conditions is..."	"Improves performance universally"
Validated causal claim	"The preregistered intervention supports this claim after controlling stated alternatives"	"Perfect attribution"

Appendix C: Normative Release Checklist

Every versioned release MUST record the results of the following specification checks. Any failing item MUST be listed as a known limitation in the Release notes; silence or an unfinished checklist is not an acceptable substitute:

1. Chinese and English titles, section numbers, figure numbers, Replay names, and terminology align.
2. Schemas accept every valid example and reject the versioned representative invalid examples.
3. Every figure has a source, prompt or editable source, content digest, and license record.
4. PDF fonts, links, pagination, captions, and text extraction pass release checks.
5. Repository secret scanning and asset review confirm that the release contains no credentials, personal data, or unauthorized third-party material.
6. External adapter cases such as DSH record an immutable commit, nearest-tag relationship, and verification date.
7. Release artifacts provide SHA-256 digests and trace to an immutable Git tag.
8. Public Draft status, license scope, trademark boundary, canonical repository, and Issues entry point are visible on the repository home page.

References

1. Creative Commons, [Attribution 4.0 International](#), accessed August 26, 2026.
2. Apache Software Foundation, [Apache License, Version 2.0](#), accessed August 26, 2026.
3. IETF, [RFC 8785: JSON Canonicalization Scheme](#), 2020.
4. OpenTelemetry, [Semantic Conventions for Generative AI Systems](#), pinned to commit 56d6b11a02129319bf371083fa134b7ce989c976, accessed August 26, 2026; the repository was still evolving at that date.
5. Semantic Versioning, [Semantic Versioning 2.0.0](#), accessed August 26, 2026.
6. WIPO, [Copyright](#), accessed August 26, 2026.
7. DeepSeek Harness, [project repository](#); see Section 15 for the verified commit.

Revision Notes

Version 0.4 preserves the original vision of v0.3 while making the following structural corrections: renaming [Body](#) to [Blueprint](#) in ABIR; establishing ABIR and [AssemblyRecipe](#) as the canonical structural record and the two-dimensional blueprint as its authoritative editing projection; separating [structuralProvenance](#) from composable field-level [operationCapabilities](#); separating event storage, origin, and replay roles; limiting fixed-model-stream use to contract regression; adding statistical discipline, inverse operations, external-side-effect compensation, drift controls, security, supply-chain controls, accessibility, and bilingual governance; and unifying the implementation roadmap and conformance levels.

The original v0.3 source is preserved intact in [archive/v0.3/](#) to document authorship and the evolution of the work.